

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/315909638>

Practical Geometric Modeling Using Geometric Algebra Motors

Article in *Advances in Applied Clifford Algebras* · September 2017

DOI: 10.1007/s00006-017-0777-z

CITATIONS

2

READS

1,165

2 authors:



[Mauricio Cele Lopez Belon](#)

Independent

5 PUBLICATIONS 6 CITATIONS

[SEE PROFILE](#)



[Dietmar Hildenbrand](#)

Technische Universität Darmstadt

75 PUBLICATIONS 836 CITATIONS

[SEE PROFILE](#)

Some of the authors of this publication are also working on these related projects:



Data Models [View project](#)



Conformal Geometric Algebra Mesh Deformation [View project](#)

Practical geometric modeling using Geometric Algebra motors

Mauricio Cele Lopez Belon and Dietmar Hildenbrand

Abstract. The usage of Geometric Algebra motors instead of Euclidean vectors for describing the position and orientation of points on a surface has promising applications in Computer Science and Engineering. Common geometric transformations, such as rotations and translations of Euclidean points, are also applicable to motors. However, encoding vertex positions and orientations as motors adds the capability of computing *motor interpolation* on surfaces. Thanks to that, general curves and surfaces can be generated by a motor interpolation process using different basis functions and parameterizations. In applications, the generated surfaces can be visually manipulated and deformed in a predictable way by changing the motors. In this paper we look inside the theory behind those applications as well as practical details on how Geometric Algebra algorithms can be computed efficiently. We show that geometric deformations can be computed at interactive rates on surface models with millions of vertices using the GPU.

Mathematics Subject Classification (2010). Parallel algorithms 68W10; Clifford algebras, spinors 15A66.

Keywords. Conformal Geometric Algebra, Mesh Deformation, Motor Interpolation, Geometric Modeling, Conformal Mapping.

1. Introduction

Parametric representations of surfaces such as Free-Form Deformation (FFD) [16] and Harmonic Coordinates [9] are widely used in geometric modeling applications. In those representations, a surface is parameterized inside a very coarse lattice of points (or a cage) in 3D space known as Domain. From the parameterization the surface can be reconstructed by interpolating the Domain points using a set of real-valued basis functions. The translation and rotation of the Domain points produces the deformation of the parametrized surface.

In this work we present a comprehensive review of how surfaces can be represented with Conformal Geometric Algebra motors as parametric Domain [10], its applications to surface deformation and its efficient computation.

In practice, computations carried out in large surface models are challenging. We have to address the practical issues of translating multivector equations to a form that can be efficiently computed in graphics hardware. We provide a method to compute deformations at large scale. Runtime comparisons show that our method is able to process models with millions of vertices and faces.

1.1. Conformal Geometric Algebra

Conformal Geometric Algebra (CGA), introduced by Hestenes, Li and Rockwood in [5], is a model of Geometric Algebra tailored for doing conformal geometry. A thorough introduction to CGA with emphasis in Computer Science and Engineering can be found in [3, 6, 13].

In CGA the Euclidean space $\mathbb{R}^3$ is extended with two additional basis vectors $\{e_o, e_\infty\}$ (also called Minkowski basis), defined as *null* vectors: $e_o^2 = e_\infty^2 = 0$ with inner product $e_o \cdot e_\infty = -1$ and orthogonal to vectors in $\mathbb{R}^3$. The vector e_o can be interpreted as the point at the origin and the vector e_∞ as the point at infinity. A general null vector p can be constructed in the 5D basis $\{\mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_3, e_o, e_\infty\}$ given a Euclidean vector $\mathbf{p}$ as:

$$p = e_o + \mathbf{p} + \frac{\|\mathbf{p}\|^2}{2} e_\infty \quad (1.1)$$

We call p a *conformal* point (a sphere of radius zero) denoting only a location in space. Note that if $\mathbf{p}$ is at the origin then the conformal point at $\mathbf{p}$ is $p = e_o$. Hence we regard e_o also as a conformal point.

Rigid body transformations in the Euclidean space $\mathbb{R}^3$ are represented as orthogonal transformations in Conformal Geometric Algebra known as *motors*. A motor M can be generated as the geometric product of an even number of reflection planes. A reflection of any k -blade A in a plane $\pi = \mathbf{n} + de_\infty$ with normal $\mathbf{n}$ is expressed as the sandwich product $(-1)^k \pi A \pi$. A translation motor $T_{\mathbf{v}}$ with direction vector $\mathbf{v}$ is defined as the product of two *parallel* planes $(\mathbf{n} + \frac{d_1}{2} e_\infty)$ and $(\mathbf{n} + \frac{d_2}{2} e_\infty)$ with normal vector $\mathbf{n} = \frac{\mathbf{v}}{\|\mathbf{v}\|}$ separated from each other by vector $\mathbf{v} = \frac{1}{2}(d_1 - d_2)\mathbf{n}$. A rotation motor $R_{\mathbf{B}\theta}$ is defined as the product of two *incident* planes at the origin $\mathbf{n}_1$ and $\mathbf{n}_2$ with dihedral angle of $\frac{\theta}{2}$ and with rotation bivector $\mathbf{B} = \frac{\mathbf{n}_1 \wedge \mathbf{n}_2}{\|\mathbf{n}_1 \wedge \mathbf{n}_2\|}$. A general motor M is defined as the geometric product of a translation and a rotation $M = T_{\mathbf{v}} R_{\mathbf{B}\theta}$. Motors act on all CGA entities $\{A\}$ in the same way using the sandwich product $A' = M A \tilde{M}$, where $\tilde{M}$ is the *reverse* of M and can be obtained by reversing the order of all the products of reflection planes.

Every motor can be written in terms of the exponential of a bivector B as $M = e^{-B}$. The bivector B is the generator of the motion, and is an element of the Lie algebra of the SE(3) Lie group.

For the case of rotations, the Euclidean bivector $\mathbf{B}$ defines the plane of rotation and by convention it has unit norm, so it can be multiplied by half

the angle of rotation $\frac{\theta}{2}$. A rotation $R_{\mathbf{B}\theta}$ in a plane with attitude $\mathbf{B}$ and angle of θ radians has the exponential form $R_{\mathbf{B}\theta} = e^{-\mathbf{B}\frac{\theta}{2}}$:

$$e^{-\mathbf{B}\frac{\theta}{2}} = \cos\left(\frac{\theta}{2}\right) - \mathbf{B} \sin\left(\frac{\theta}{2}\right)$$

On the other hand, a translation $T_{\mathbf{v}}$ with direction vector $\mathbf{v}$ has the exponential form $T_{\mathbf{v}} = e^{-\frac{1}{2}\mathbf{v} \wedge e_{\infty}}$. Here the bivector $B = \mathbf{v} \wedge e_{\infty}$ contains the point at infinity and so can be regarded as a *flat* object, analogous to a circle that contains the point at infinity which is isomorphic to a flat line. So the bivector B in this case can be interpreted as a kind of direction element.

$$e^{-\frac{1}{2}\mathbf{v} \wedge e_{\infty}} = 1 - \frac{1}{2}\mathbf{v} \wedge e_{\infty}$$

Notice that the application of a translator $T_{\mathbf{v}}$ to the point at the origin e_o produces a conformal point p as expected:

$$T_{\mathbf{v}} e_o \tilde{T}_{\mathbf{v}} = e_o + \mathbf{v} + \frac{1}{2}\|\mathbf{v}\|^2 e_{\infty} \quad (1.2)$$

Notice also that combining translators is equivalent to adding their direction vectors $T_{\mathbf{w}} T_{\mathbf{v}} = 1 - \frac{1}{2}(\mathbf{w} + \mathbf{v}) \wedge e_{\infty}$, and combining with the inverse translator is equivalent to subtract their direction vectors $T_{\mathbf{w}} \tilde{T}_{\mathbf{v}} = T_{\mathbf{w}-\mathbf{v}}$.

A translated rotation $R_{T_{\mathbf{v}}B\tilde{T}_{\mathbf{v}}} = T_{\mathbf{v}} R_{\mathbf{B}\theta} \tilde{T}_{\mathbf{v}}$ is a rotation shifted from the origin by the vector $\mathbf{v}$. That means that the axis of rotation does not pass through the origin. The exponential for this rotor is $R_{T_{\mathbf{v}}B\tilde{T}_{\mathbf{v}}} = e^{\Lambda\theta/2}$, where the bivector $\Lambda = T_{\mathbf{v}}\mathbf{B}\tilde{T}_{\mathbf{v}}$ represents a (dual) line. Let us recall the Chasles' theorem:

Theorem 1.1. Chasles' theorem: *a general rigid body transformation can be represented by the motor M :*

$$M = T_{\mathbf{w}} T_{\mathbf{v}} R_{\mathbf{B}\theta} \tilde{T}_{\mathbf{v}} \quad (1.3)$$

$$M = e^{-\frac{1}{2}\mathbf{w} \wedge e_{\infty} - \frac{1}{2}T_{\mathbf{v}}\mathbf{B}\theta\tilde{T}_{\mathbf{v}}} \quad (1.4)$$

where $T_{\mathbf{w}}$ and $T_{\mathbf{v}}$ are translations and $R_{\mathbf{B}\theta}$ is a rotation. The translation vector $\mathbf{v}$ is in the $\mathbf{B}$ plane, so that $\mathbf{v} \wedge \mathbf{B} = 0$, and $\mathbf{w}$ is perpendicular to $\mathbf{B}$ plane (so that $\mathbf{w} \cdot \mathbf{B} = 0$). The incremental application of M produces a helical motion with a translation of $\mathbf{w}$ along the axis $\Lambda = T_{\mathbf{v}}\mathbf{B}\tilde{T}_{\mathbf{v}}$ and a rotation of θ radians around that axis.

1.2. Motor interpolation

Given two motors M_1 and M_2 their interpolation function $M(\lambda)$ is given by $M(\lambda) = M_1(\tilde{M}_1 M_2)^{\lambda}$ where $\lambda \in [0, 1]$ ([3]). This form of interpolation presents some difficulties that are hard to overcome. Its evaluation is not computationally efficient as for the interpolation of n motors we need to compute at least n geometric products. Also it is non linear which is inconvenient for solving numerical problems (section 2) and precludes the formulation of the tensor product of motor-splines (section 3).

In order to overcome these difficulties we have to linearize the formulation. The naive linearization $M(\lambda) \approx (1 - \lambda)M_1 + \lambda M_2$ works only when M_1 and M_2 are “close” to each other in the tangent space of the manifold. Moreover, it would work only for rotations or translations but not both since it would not produce a proper motor. In contrast, motor logarithms can be linearly interpolated since they are in the 6D linear tangent space of generators of rigid transformations and the exponential of any of the interpolated bivectors gives a valid intermediate motor:

$$M(\lambda) = \exp((1 - \lambda) \log(M_1) + \lambda \log(M_2)) \quad (1.5)$$

The bivector interpolation can be evaluated efficiently and its linearity allow us to express interpolation as a linear matrix system. This form of interpolation was used by Wareham *et. al.* [17, 18] and Lopez [10] in geometric modeling applications. The interpolation of pure rotors and translators given by (1.5) is linear in the rotation angles and translation vectors respectively, unfortunately the interpolation of general motors given by (1.5) is not linear in the angle. However this was not a serious limitation in our applications.

The motor logarithm can be found using Chasles’ theorem (1.1) following the procedure described in [3].

$$M = T_{\mathbf{v}} R_{\mathbf{B}\theta} \quad (1.6)$$

$$\log(M) = \frac{1}{2}(-(\mathbf{v} \wedge \mathbf{B})\mathbf{B}^{-1} + (1 - R_{\mathbf{B}\theta}^2)^{-1}(\mathbf{v} \cdot \mathbf{B}\theta))e_{\infty} - \mathbf{B}\frac{\theta}{2} \quad (1.7)$$

where the following values can be retrieved from M as follows:

$$\begin{aligned} R_{\mathbf{B}\theta} &= -e_o \cdot (M e_{\infty}) , \quad \mathbf{v} = -2(e_o \cdot M) R_{\mathbf{B}\theta}^{-1} \\ \mathbf{B} &= \langle R_{\mathbf{B}\theta} \rangle_2 / \|R_{\mathbf{B}\theta}\| , \quad \theta = -2 \arctan \left(\frac{\langle R_{\mathbf{B}\theta} \rangle_2 \mathbf{B}^{-1}}{\langle R_{\mathbf{B}\theta} \rangle_0} \right) \end{aligned}$$

2. Motor curves

A simple application of motor interpolation (1.5) is curve and surface representation. Traditionally, unit quaternions are used in order to control rotations of 3D objects, virtual cameras and robots, because they are free from singularities such as the “Gimbal lock”. When objects change their position and their orientation at the same time both changes have to be smooth and natural. The smooth motion of positions can be represented as a curve in $\mathbb{R}^3$. The smooth change in orientations can be represented as a quaternion curve. Making both curves match a set of unified modeling constraints smoothly is difficult. We extend a family of parametric curves defined in $\mathbb{R}^3$ to curves defined by motors in conformal space. Such motor curves interpolate the required positions and orientations changing both gradually at equal measure. This is essentially a motor fitting problem. Let us first define the motor curve:

Definition 2.1. *Given a sequence of motors $\{M_i\}_{i=1}^n$, called control motors, and a sequence of basis functions $\{B_i\}_{i=1}^n$ such that $\sum_{j=1}^n B_j(\lambda) = 1$, $\lambda \in$*

$[0, 1]$ the motor curve is given by $p(\lambda) = M(\lambda)p_0\tilde{M}(\lambda)$ where:

$$M(\lambda) = \exp \left(\sum_{j=1}^n B_j(\lambda) \log(M_j) \right) \quad (2.1)$$

and p_0 is an arbitrary conformal point.

The weight functions $B_j(\lambda)$ could be, for instance, the B-spline weights. The choice of p_0 is arbitrary; in the experiments presented here we used $p_0 = e_o$ but ultimately the choice of origin depends on applications. To visualize orientations we define the functions $t_1(\lambda)$, $t_2(\lambda)$ and $t_3(\lambda)$ which simply apply the motor $M(\lambda)$ to the *tangent* vectors $e_o \wedge \mathbf{e}_1$, $e_o \wedge \mathbf{e}_2$ and $e_o \wedge \mathbf{e}_3$ respectively. Tangent vectors are a combination of position and direction of the form $e_o \wedge \mathbf{v}$. Applying a rotation to a tangent vector only affects its direction $R(e_o \wedge \mathbf{v})\tilde{R} = e_o \wedge R\mathbf{v}\tilde{R}$, while applying a translation only affects its location $T_{\mathbf{p}}(e_o \wedge \mathbf{v})\tilde{T}_{\mathbf{p}} = p \wedge \mathbf{v} + (\mathbf{p} \cdot \mathbf{v})p \wedge e_\infty$ where we can recognize $p \wedge e_\infty$ as a flat point denoting the translated position while direction $\mathbf{v}$ remain unchanged.

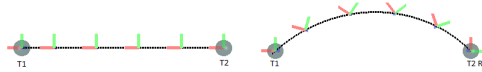


FIGURE 1. Motor interpolation. Left: interpolation of translations. Right: Interpolation of a rotation bends the line

The produced curves have certain particularities. When no rotation is specified, the curve reduces to the curve in $\mathbb{R}^3$, so the curve is defined only inside the convex hull of the control motors. When rotation is incorporated the curve changes its curvature by some amount proportional to the amount of rotation added. This is due to the coupling of rotations and translations. To understand this better, let us define the translators $T_1 = e^{-\frac{1}{2}\alpha e_{1\infty}}$ and $T_2 = e^{-\frac{1}{2}\beta e_{1\infty}}$. The interpolation of the motors using (1.5) gives us a line segment along $\mathbf{e}_1$ direction since $M(\lambda) = e^{-\frac{1}{2}e_{1\infty}((1-\lambda)\alpha + \lambda\beta)}$. Then incorporating a rotation R_2 in the plane $\mathbf{e}_{12}$ to T_2 we get $T_2 R_2 = e^{-\frac{1}{2}\beta e_{1\infty}} e^{-\frac{1}{2}\theta e_{12}}$ we expect to see a term in the logarithm of motor $T_2 R_2$ that generate this curvature. Taking the logarithm (1.7) we get $\log(T_2 R_2) = -\frac{1}{4}\beta\theta \cot(\frac{\theta}{2})e_{1\infty} - \frac{1}{2}\theta e_{12} - \frac{1}{4}\theta\beta e_{2\infty}$. Of course the last term $-\frac{1}{4}\theta\beta e_{2\infty}$ is involving a direction orthogonal to the direction of motion $\mathbf{e}_1$ generating the curvature and the amount is proportional to $-\frac{1}{4}\theta\beta$. This can be seen in the Figure 1.

As pointed out in [2] the spline interpolation of bivectors leads to a not covariant motion under a change of the interpolants i.e., moving the target motors and then interpolating is not the same as moving the interpolated sequence. The reason given in [2] is that the $SE(3)$ group cannot be endowed with a “bi-invariant Riemannian metric”. That means that interpolated motors cannot be covariant simultaneously under changes of body frame and of world frame. Two practical solutions to this dilemma are compared in [1].

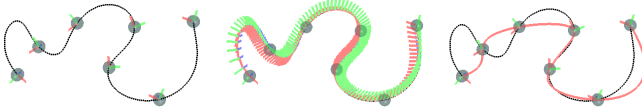


FIGURE 2. Example of B-Spline interpolation. Left: interpolated curve of translations. Middle: Interpolated curve of translations and rotations. Right: Comparison with the Euclidean B-spline interpolation (red). Both curves pass through the constrained positions but Motor curve is adapted to interpolate orientations.

Now we show how the B-spline interpolation can be solved. We look for the sequence of motors $\{M_j\}_{j=1}^n$ that generate the B-spline motion passing through the motors $\{Q_i\}_{i=1}^n$. In other words our goal is, having the sequence of motors $\{Q_i\}_{i=1}^n$ as input, to determine the sequence of motors $\{M_j\}_{j=1}^n$ such that

$$\exp \left(\sum_{j=1}^n \mathcal{B}_j(\lambda_i) \log(M_j) \right) = Q_i \quad (2.2)$$

where $\mathcal{B}_j(\lambda_i)$ are the cubic B-spline weights. This problem can be solved using linear least squares fitting as follows. Taking the logarithm on both sides of (2.2) and rewriting it in matrix form we get:

$$B M = Q \quad (2.3)$$

where the column vector Q is $\{\log(Q_i)\}$, the column vector M is $\{\log(M_j)\}$ and the matrix $B_{m \times n}$ is:

$$B_{m \times n} = \begin{bmatrix} \mathcal{B}_1(\lambda_1) & \mathcal{B}_2(\lambda_1) & \cdots & \mathcal{B}_n(\lambda_1) \\ \mathcal{B}_1(\lambda_2) & \mathcal{B}_2(\lambda_2) & \cdots & \mathcal{B}_n(\lambda_2) \\ \vdots & \vdots & \ddots & \vdots \\ \mathcal{B}_1(\lambda_m) & \mathcal{B}_2(\lambda_m) & \cdots & \mathcal{B}_n(\lambda_m) \end{bmatrix} \quad (2.4)$$

The curve fitting literature points out that when the number of motors $\{M_j\}$ is less than the number of “samples” $\{Q_j\}$ the curve is smoother. In our examples the number of samples is low so the direct interpolation gives smooth curves as well. The best solution (in general) is obtained by solving the LLS normal equations:

$$(B^T B) M = B^T Q \quad (2.5)$$

The matrix $(B^T B)$ can be ill-conditioned depending on the chosen *knot* vector of the B-spline basis. If that is the case the system can be solved by SVD. The final step is taking the exponential of the bivectors to get the required $\{M_j\}$. An example of motor curve fitting can be seen in Figure 2, this curve can be defined outside the convex hull of motors and can interpolate smoothly positions and orientations.

3. Motor FFD

The Free Form Deformation (FFD) [16] is a parameterization technique used extensively in geometric modeling and shape optimization for example in the parametrization of airfoils and wings, for potential flows, thermal and viscous flows and cardiovascular devices [15]. The FFD parameterization need the definition of a lattice of control points $\mathbf{P}_{i,j,k}$ which defines a volume Ω with the topology of a cube. Given a surface represented by a mesh $\mathcal{M} = \{\mathcal{P}, \mathcal{G}\}$ where $\mathcal{P}$ is the set of m points $\mathcal{P} = \{p_i\}_{i=1}^m$ and $\mathcal{G}$ is the connectivity graph, $\mathcal{M}$ is embedded in the Domain Ω by the invertible map $\Psi : \mathcal{P} \rightarrow (u, v, w) \in [0, 1] \times [0, 1] \times [0, 1]$. An arbitrary lattice corner $\mathbf{P}_0 = \mathbf{P}_{0,0,0}$ is fixed as origin of coordinates and all points of Ω can be defined in terms of $\mathbf{P}_0$ and three orthogonal vectors $\mathbf{u}, \mathbf{v}$ and $\mathbf{w}$ as $\mathbf{P}_{u,v,w} = \mathbf{P}_0 + u\mathbf{u} + v\mathbf{v} + w\mathbf{w}$. The parameter coordinates of a point $\mathbf{p}$ can be found as follows:

$$u = \frac{(\mathbf{p} - \mathbf{P}_0) \cdot (\mathbf{v} \wedge \mathbf{w})^*}{\mathbf{u} \cdot (\mathbf{v} \wedge \mathbf{w})^*}, \quad v = \frac{(\mathbf{p} - \mathbf{P}_0) \cdot (\mathbf{u} \wedge \mathbf{w})^*}{\mathbf{v} \cdot (\mathbf{u} \wedge \mathbf{w})^*}, \quad w = \frac{(\mathbf{p} - \mathbf{P}_0) \cdot (\mathbf{u} \wedge \mathbf{v})^*}{\mathbf{w} \cdot (\mathbf{u} \wedge \mathbf{v})^*}$$

where $\mathbf{X}^* = \mathbf{X} \cdot \tilde{\mathbf{I}}_3$ and $\mathbf{I}_3 = \mathbf{e}_{123}$ is the Euclidean pseudoscalar. The points of the mesh $\mathcal{M}$ embedded within the lattice are reconstructed by defining the following mapping from the lattice to the object.

$$\mathbf{p}_l = \sum_{i=1}^n \sum_{j=1}^m \sum_{k=1}^r \mathcal{B}_i(u^l) \mathcal{B}_j(v^l) \mathcal{B}_k(w^l) \mathbf{P}_{i,j,k} \quad (3.1)$$

where $\mathcal{B}_i, \mathcal{B}_j$ and $\mathcal{B}_k$ are Bézier or B-Spline basis functions, $\mathbf{P}_{i,j,k}$ are the lattice control points and $\{u^l, v^l, w^l\}$ is the parameterization of the point $\mathbf{p}_l$. Mesh deformation is achieved by translating the control points. FFD does not handle well large rotations of control points, in some cases it is not possible to produce convincing deformations when they involve large rotations or twists. For instance, the “candy wrapper” effect is produced (see Figure 3) when a large twist is applied to the control lattice. More visual comparisons can be found in Lopez [10].

In the Geometric Algebra version, the control points $\mathbf{P}_{i,j,k}$ of the lattice are replaced by motors $R_{i,j,k}$. Initially motors are only translations $R_{i,j,k} = e^{-\frac{1}{2}\mathbf{P}_{i,j,k}e_\infty}$ which can reconstruct the mesh vertices with the mapping $p_l = R_{u,v,w}e_o\tilde{R}_{u,v,w}$, where $R_{u,v,w}$ is an interpolated motor. The choice of origin is arbitrary, in our experiments we use e_o with the lattice centered at it. The motor $R_{u,v,w}$ is defined as:

$$R_{u,v,w} = \exp \left(\sum_{i=1}^n \sum_{j=1}^m \sum_{k=1}^r \mathcal{B}_i(u) \mathcal{B}_j(v) \mathcal{B}_k(w) \log(R_{i,j,k}) \right) \quad (3.2)$$

Traditional FFD deformation is obtained applying translations T_v to individual control motors. Applying rotations R as produce curved shapes with circular rotations and screws. We can see the comparison of this technique with FFD in Figure 3.

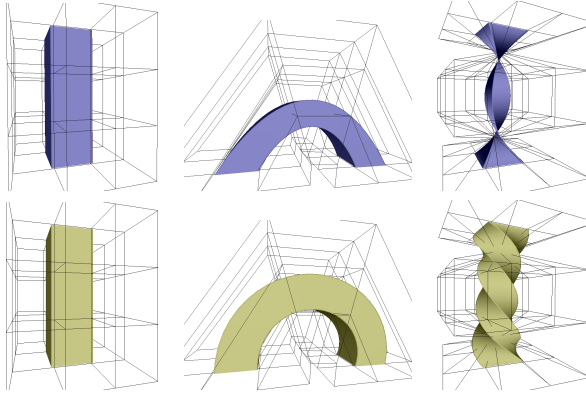


FIGURE 3. Comparison of motor-based FFD (yellow) and FFD (blue). Left: the reference bar. Middle: Only motor-based FFD could produce circular bending. Right: Under large twisting the motor-based FFD produce correct results while FFD present the “candy wrapper” effect.

4. Spline Aligned Surfaces

The seminal work of Pinkall and Polthier [14] shows how to interpolate given data over a discrete domain using a conformal mapping. Conformal mappings are harmonic, and harmonic maps are critical points of the Dirichlet energy (stretching energy) $\int_{\Omega} |\nabla f|^2 dA$, which generate minimal surfaces [14]. The *discrete harmonic energy* [14] of a map f defined on mesh vertices $\{v_i\}$ has the form:

$$E(f) = \sum_{i,j} w_{ij} |f(v_j) - f(v_i)|^2 \quad (4.1)$$

where $|\cdot|$ is the L^2 norm and w_{ij} are the (symmetric) cotangent weights [14] defined on triangle edges going from v_i to v_j . The discrete Laplacian operator can be identified in that energy as:

$$\Delta f = \sum_{j \in N_i} w_{ij} (f(v_j) - f(v_i)) \quad (4.2)$$

where N_i is the set of indices of the neighbors of vertex v_i . It is known that harmonic energy minimizes angular distortions, thus conformal mappings are harmonic. That means that harmonic functions smoothly blend boundary conditions over the domain. Harmonic functions are intrinsic to surfaces and independent of the discretization used to produce meshes. Following [14], Zayer *et. al.* [19] proposed the interpolation of unit quaternions over a mesh using harmonic functions. The harmonic quaternion interpolation over a mesh can be formulated as the solution to the following discrete harmonic equation:

$$\sum_{j \in N_i} w_{ij} (Q_j - Q_i) = 0 \quad (4.3)$$

subject to Dirichlet boundary conditions $Q_i = q_i$, where $\{Q_i\}$ are unit quaternions and $\{w_{ij}\}$ are the cotangent weights. This leads to the solution of a sparse linear matrix system. This interpolation produces a smooth field of quaternions by “averaging” the boundary conditions gradually over the surface. This is in effect equivalent to a linear interpolation of boundary conditions across the surface. A simpler interpolation of motors on a surface can be done by using a Harmonic *scalar field* which is computed on the surface with Dirichlet boundary conditions set to 0 and 1 for two disjoint groups of vertices. The discrete Laplace equation over the reals has the form:

$$\sum_{j \in N_i} w_{ij}(\lambda_j - \lambda_i) = 0 \quad (4.4)$$

subject to Dirichlet boundary conditions $\lambda_i = 0$ and $\lambda_i = 1$ for two disjoint groups of vertices named *handles*. The harmonic field can be directly used for motor interpolation $R_i = e^{(1-\lambda_i) \log(R_1) + \lambda_i \log(R_2)}$ where motor R_1 is attached to the handle with $\lambda_i = 0$ and R_2 is attached to the other handle. The interpolation reduces to $M(0) = R_1$ and $M(1) = R_2$ for vertices at handles, so those vertices are transformed rigidly by the control motors, the rest of vertices are transformed by a local blend of control motors. Since we already have defined in (2.1) a spline interpolation for an arbitrary sequence of control motors, the scalar harmonic field can also be used to produce spline-aligned surfaces.

Let $\mathcal{R} = \{R_j\}_{j=1}^n$ be a sequence of control motors and let $\{\lambda_i\}$ be a harmonic scalar field computed over the mesh $\mathcal{M}$ such that $0 \leq \lambda_i \leq 1$. The spline function $S(\lambda)$ is defined as $S(\lambda_i) = e^{\sum_{j=1}^n B_j(\lambda_i) \log(R_j)}$ where $B_j(\lambda_i)$ is a sequence of basis functions forming a partition of unity $\sum_j B_j(\lambda_i) = 1$. If we see the harmonic field $\{\lambda_i\}$ as a real valued function $\lambda : \mathbb{R}^3 \mapsto \mathbb{R}$ then the spline function $S(\lambda)$ can be viewed as producing a motor M_i for a given mesh point $\mathbf{p}_i \in \mathbb{R}^3$ in the sense that $M_i = S(\lambda(\mathbf{p}_i))$. Details on how to compute the harmonic field λ for a mesh $\mathcal{M}$ (or region of interest) can be found in Lopez [10].

Equipped with the spline function S and the harmonic scalar field $\{\lambda_i\}$ we define the deformation function f of the form $f : \mathcal{M} \mapsto \mathcal{M}$ which maps a mesh $\mathcal{M}$ into a deformed version $\mathcal{M}'' = \{p'' = f(p) \mid p \in \mathcal{P}, p'' \in \mathcal{P}''\}$ as follows:

$$f(p_i) = M_i p_i \tilde{M}_i \quad (4.5)$$

where $M_i = S(\lambda(\mathbf{p}_i))$ is called the *deforming* motor and p_i is the conformal point constructed from the mesh point $\mathbf{p}_i$. As with FFD, multiplying control motors with translations and rotations produces the deformation of the surface parameterized by the harmonic scalar field. An example of spline-aligned surface can be seen in Figure 4.

We now show how the deforming function f produces the identity mesh. Let us define the inverse deformation function f^{-1} as $f^{-1}(p_i) = \tilde{M}_i p_i M_i$, so that $f^{-1}(f(p_i)) = \tilde{M}_i(M_i p_i \tilde{M}_i)M_i = p_i$. Then we define the inverse image of the mesh vertices as the set $\mathcal{P}' = \{p'_i = f^{-1}(p_i) \mid p \in \mathcal{P}\}$. Then, the

original set of points $\mathcal{P}$ is obtained from the inverse image $\mathcal{P}'$ by applying the deforming function to it $\mathcal{P} = \{p_i = f(p'_i) | p' \in \mathcal{P}'\}$. In applications it is convenient to precompute the set $\mathcal{P}'$ from the undeformed mesh model, so the undeformed set $\mathcal{P}$ (identity mesh) is obtained as output of the first deformation, then further deformations can be obtained by rotating an moving the control motors.

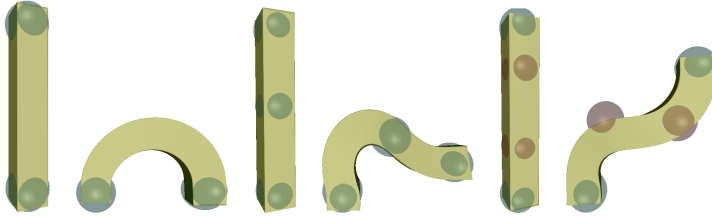


FIGURE 4. Spline Aligned Surfaces: 3D bar is deformed similarly to motor curves. Spheres are used to group vertices in *handles*, the control motors are located at the sphere centers. From left to right: Bending is achieved by interpolating two control motors. Deformation similar to a quadratic curve is achieved interpolating three control motors. The cubic case is shown at the right. As with cubic Bézier curves, the curve is not passing through middle control motors (in red).

5. Geometric Algebra in Practice

The first time Geometric Algebra was introduced to a wider computer graphics audience, was through some courses at the SIGGRAPH conferences in 2000 and 2001 (see [11]) and the Eurographics 2004 [8]. The main disadvantage of Geometric Algebra at this time was, that the clear structure and greater elegance resulted in lower runtime performance. For this reason Geometric Algebra applications are often reformulated as matrix algebra which has efficient methods and libraries for computing on large amounts of data. We present a method to compute geometric deformations of high resolution models, with millions of vertices, using the motor interpolation described in sections 3 and 4 without switching back to linear matrix algebra, thus carrying out computations using Geometric Algebra.

Code optimizers such as Gaalop [7] are becoming key for decoupling algebraic formulations from code. Engineers can focus on defining correct Geometric Algebra operations instead of focusing on transforming Geometric Algebra operations to specific code tailored for efficient GPU or CPU execution. Since all of the results of Geometric Algebra operations on multivectors are multivectors, Gaalop *symbolically* computes and simplifies the resulting multivectors in order to determine which of the coefficients are actually needed and what is the simplest expression for each coefficient. Our

approach is to describe the motor interpolation in the Geometric Algebra programming language CLUCalc [12] then optimize it using Gaalop. Generated code by Gaalop is integrated directly with the production code base. Gaalop compiles the code to an intermediate representation using either a Maple-based or a table-based approach. Based on this representation, implementations for different sequential and parallel platforms can be derived, see Figure 5.

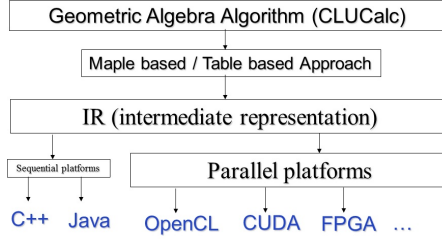


FIGURE 5. The Geometric Algebra Computing architecture. Algorithms are compiled to an intermediate representation for compilation to different computing platforms.

Motor interpolation can be carried out in parallel for each mesh vertex. This type of parallelism is known as “data parallelism” since the same code is executed on different pieces of data. For motor interpolation, no communication or synchronization among threads is needed which simplifies further the code. Gaalop is able to generate code directly for GPU languages such as CUDA and OpenCL, however for the sake of simplicity we generate C++ code. C++ code can be “run” directly in GPU using the recently introduced *C++ AMP* (C++ Accelerated Massive Parallelism) technology for GPGPU computing.

Our implementation is as follows. We store in GPU memory all the data as arrays: vertex positions $\{p_i\}$, vertex normals $\{n_i\}$, spline weights $\{w_i \mid w_i = \mathcal{B}_j(\lambda_i)\}_{i=1}^n$ where $\{\mathcal{B}_j(\lambda_i)\}$ is a sequence of basis functions and the bivectors $\{B_i \mid B_i = \log(R_i)\}$ where $\{R_i\}$ is a sequence of control motors. The GPU-kernel to be executed is then the expression (5.1):

$$\begin{aligned} p_i &= M_i p'_i \tilde{M}_i \\ n_i &= M_i n'_i \tilde{M}_i \end{aligned} \quad (5.1)$$

where M_i is the interpolated motor computed as:

$$M_i = \exp \left(\sum_{j=1}^n w_j B_j \right) \quad (5.2)$$

Gaalop generates efficient expressions for the geometric products and the bivector exponential. The bivector exponential was approximated with the first five terms of the power series $e^B \approx 1 + B + \frac{B^2}{2} + \frac{B^3}{3!} + \frac{B^4}{4!}$. We use the CluCalc code of Listing 1 for generating the C++ AMP kernel used in our experiments.

LISTING 1. CluCalc code for Gaalop Optimizer

```

// Power series approximation of the exponential
Exp_approx = {
  1 + _P(1) + _P(1)*_P(1)/2 + _P(1)*_P(1)*_P(1)/6
    + _P(1)*_P(1)*_P(1)*_P(1)/24
}
// Definition of the input bivectors B0, B1, B2 and B3
B0 = B00*(e1^e2) + B01*(e1^e3) + B02*(e2^e3) +
    B03*(e1^einf) + B04*(e2^einf) + B05*(e3^einf);
B1 = B10*(e1^e2) + B11*(e1^e3) + B12*(e2^e3) +
    B13*(e1^einf) + B14*(e2^einf) + B15*(e3^einf);
B2 = B20*(e1^e2) + B21*(e1^e3) + B22*(e2^e3) +
    B23*(e1^einf) + B24*(e2^einf) + B25*(e3^einf);
B3 = B30*(e1^e2) + B31*(e1^e3) + B32*(e2^e3) +
    B33*(e1^einf) + B34*(e2^einf) + B35*(e3^einf);
// Compute the linear combination of bivectors
?Bi = w0*B0 + w1*B1 + w2*B2 + w3*B3;
// Get a motor through the exponential map
?Mi = Exp_approx(Bi);
// Definition of the conformal point
Pi_ = p0*e1 + p1*e2 + p2*e3 + p3*einf + 1.0*e0;
// Definition of the normal vector
Ni_ = n0*e1 + n1*e2 + n2*e3;
// Compute the deformed position
?Pi = Mi * Pi_ * (~Mi);
// Compute the deformed normal
?Ni = Mi * Ni_ * (~Mi);

```

Expressions beginning with “?” are generated as output C++ code by Gaalop. Figure 6 illustrates the power of this method.



FIGURE 6. Deformation of HiDragon model (4.9 million vertices and 9.8 million faces) is tractable using Geometric Algebra. Left, undeformed model with four control motors attached. Right, resulting deformation carried out in real time by our method

5.1. Benchmarks

We have used the Gaigen2 C++ Geometric Algebra package [4] for CPU computations and Gaalop optimizer [7] for GPU computations. We have tested our implementation on large meshes with millions of vertices and faces. Table 1 shows deformation times in milliseconds. Our GPU implementation has run at interactive rates for all meshes on a AMD FX 8350 X8 4.0GHz CPU with 16GB of RAM and ATI Radeon R9 270X 2GB GPU.

Name	#Vertices	FFD	CGA-FFD	CGA-S	CGA-S GPU
HiDragon	4,949,498	993	1279	566	198
MeDragon	3,609,383	720	938	383	154
HiNeptune	2,003,932	401	522	213	103
LoNeptune	1,321,838	276	348	142	77
LoDragon	1,233,055	250	323	154	54
Raptor	1,000,080	199	257	124	44
Dragon	437,645	86	112	53	19
Angel	237,018	48	61	30	14
Armadillo	172,974	35	45	22	11
Tyrannosaur	100,002	20	26	13	7

TABLE 1. Runtime comparison of single-threaded CPU implementations of FFD vs. CGA-FFD vs. CGA-Spline in milliseconds. Time of CGA-Spline on GPU is also given in milliseconds.

6. Conclusions

Some fundamental geometric modeling methods were generalized using motors. We gave the motivations and rationale that underlie those methods and formulated them in enough detail to be put in practice by engineers and scientists, although our analysis was not exhaustive and a deeper analysis of them should be done to fully understand their potential and limits. We believe that engineers and computer scientists could see their potential and will find them useful. Hopefully they will see Geometric Algebra as a vehicle for improving geometric modeling applications and envisioning new methods as well. We have shown how to optimize Geometric Algebra algorithms to achieve good performance on large models, comparable to classic methods based on linear matrix algebra. Recent advances in Geometric Algebra Computing [6] enables new opportunities for applied Geometric Algebra. With the availability of code generators and optimizers such as Gaalop we believe that it is easier than ever to carry out Geometric Algebra computations at large scale.

References

- [1] C. Altafini. *The de casteljau algorithm on $SE(3)$* , pages 23–34. Springer London, London, 2001.
- [2] L. Dorst. *The Representation of Rigid Body Motions in the Conformal Model of Geometric Algebra*, pages 507–529. Springer Netherlands, Dordrecht, 2008.
- [3] L. Dorst, D. Fontijne, and S. Mann. *Geometric Algebra for Computer Science*. Morgan Kaufmann Publishers Inc. 2nd Revised edition, San Francisco, CA, USA, 2009.
- [4] D. Fontijne. *Efficient Implementation of Geometric Algebra*. PhD thesis, University of Amsterdam, 2007.

- [5] D. Hestenes, H. Li, and A. Rockwood. *Geometric Computing with Clifford Algebra*, chapter A Unified Algebraic Framework for Classical Geometry. Springer, 1999.
- [6] D. Hildenbrand. *Foundations of Geometric Algebra Computing*. Springer, 2013.
- [7] D. Hildenbrand, P. Charrier, C. Steinmetz, and J. Pitt. Gaalop home page. Available at <http://www.gaalop.de>, 2014.
- [8] D. Hildenbrand, D. Fontijne, C. Perwass, and L. Dorst. Tutorial geometric algebra and its application to computer graphics. In *Eurographics Conference Grenoble*, 2004.
- [9] P. Joshi, M. Meyer, T. DeRose, B. Green, and T. Sanocki. Harmonic coordinates for character articulation. In *ACM SIGGRAPH 2007 papers*, SIGGRAPH '07, New York, NY, USA, 2007. ACM.
- [10] M. C. Lopez Belon. Applications of conformal geometric algebra in mesh deformation. In *Graphics, Patterns and Images (SIBGRAPI), 2013 26th SIBGRAPI - Conference on*, pages 39–46, Aug 2013.
- [11] A. Naeve and A. Rockwood. Course 53 geometric algebra. In *SIGGRAPH Conference, Los Angeles*, 2001.
- [12] C. Perwass. The CLU home page. Available at <http://www.clucalc.info>, 2014.
- [13] C. B. U. Perwass. *Geometric algebra with applications in engineering*, volume 4 of *Geometry and Computing*. Springer, Berlin; Heidelberg, 2009.
- [14] U. Pinkall, S. D. Juni, and K. Polthier. Computing discrete minimal surfaces and their conjugates. *Experimental Mathematics*, 2:15–36, 1993.
- [15] G. Rozza, A. Koshakji, and A. Quarteroni. Free form deformation techniques applied to 3d shape optimization problems. *Communications in Applied and Industrial Mathematics*, 4(0), 2013.
- [16] T. W. Sederberg and S. R. Parry. Free-form deformation of solid geometric models. In *Proceedings of the 13th annual conference on Computer graphics and interactive techniques*, SIGGRAPH '86, pages 151–160, New York, NY, USA, 1986. ACM.
- [17] R. Wareham, J. Cameron, and J. Lasenby. Applications of conformal geometric algebra in computer vision and graphics. In *Proceedings of the 6th international conference on Computer Algebra and Geometric Algebra with Applications*, IWMM'04/GIAE'04, pages 329–349, Berlin, Heidelberg, 2005. Springer-Verlag.
- [18] R. Wareham and J. Lasenby. Mesh vertex pose and position interpolation using geometric algebra. In F. J. Perales and R. B. Fisher, editors, *Articulated Motion and Deformable Objects*, volume 5098 of *Lecture Notes in Computer Science*, pages 122–131. Springer Berlin Heidelberg, 2008.
- [19] R. Zayer, C. Rössl, Z. Karni, and H.-P. Seidel. Harmonic guidance for surface deformation. In *In Proc. of Eurographics 05*, pages 601–609, 2005.

Mauricio Cele Lopez Belon
 Buenos Aires, Argentina
 e-mail: mclopez@outlook.com

Dietmar Hildenbrand
 Hochschule RheinMain, Ruesselsheim, Germany
 e-mail: dietmar.hildenbrand@gmail.com